RegIntel: Agentic Regulatory Compliance Gap-Detection Engine

Project Description:

RegIntel is an agentic AI system that automates regulatory compliance gap detection for Indian banks and NBFCs. The platform ingests RBI, SEBI, and FEMA circular PDFs, extracts every binding obligation clause they contain, and checks each obligation against an institution's existing internal policy library to surface coverage gaps before they become penalty exposure. The system includes a PDF Ingestion module, which parses circular text and metadata using PyMuPDF; an Obligation Extractor, which identifies binding duty language ("shall", "must", "within N days") and structures it into typed clauses; a Gap Detector, which embeds obligations and policies into a local vector store and flags any obligation with no adequately similar policy; and a Penalty Scorer, which ranks every detected gap by financial exposure.

Utilizing Groq's API with the Llama 3.3-70B Versatile model orchestrated through a LangChain tool-calling AgentExecutor, RegIntel runs the four-stage audit as an agent-directed workflow rather than a fixed script — the agent decides at runtime whether to retry a thin PDF extraction, re-run obligation extraction on a suspiciously empty result, or flag a missing policy library, instead of silently failing. Built with Flask and a vanilla HTML/CSS/JS dashboard, backed by a local ChromaDB vector store, a SQLite audit log, and Groq-hosted inference, RegIntel runs entirely on free-tier infrastructure with no GPU, no paid API, and no cloud server required. With secure API key management via .env and a startup health check that validates the configured Groq model before any real audit can run, RegIntel gives compliance officers an auditable, board-ready gap report in minutes rather than weeks of manual circular review.

Scenarios

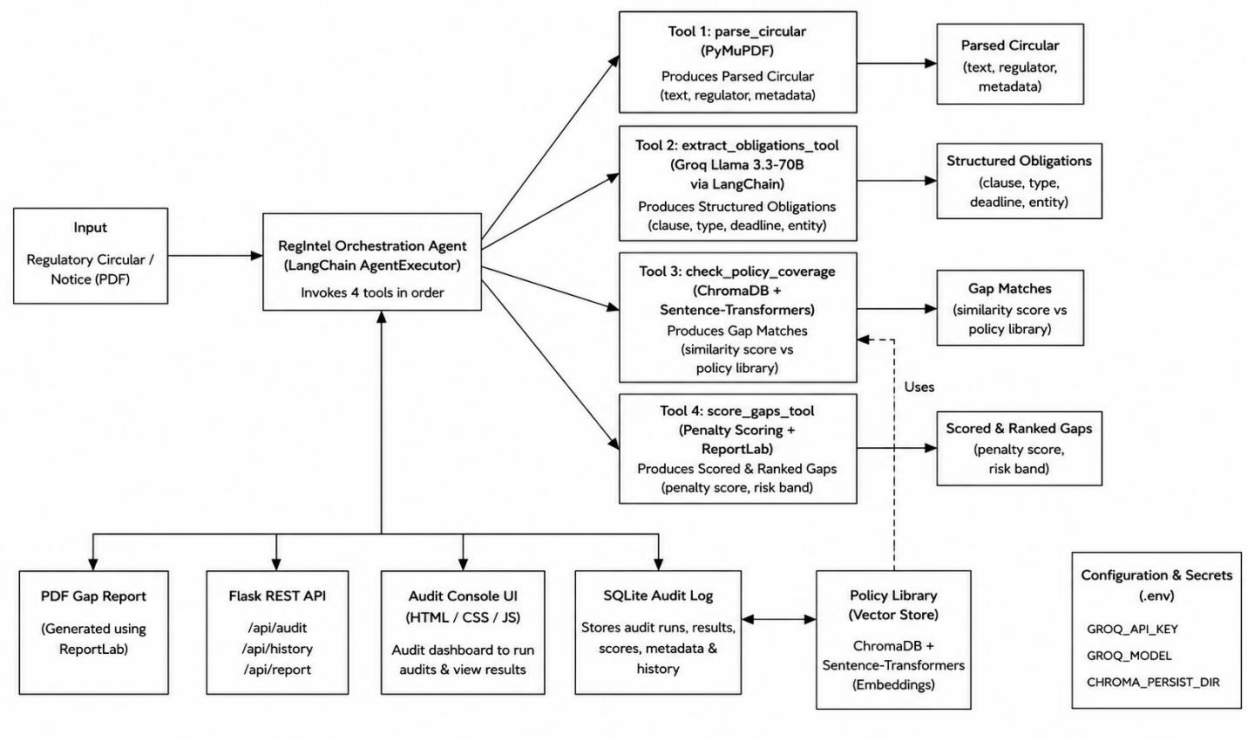
Scenario 1: A compliance officer uploads a newly issued RBI circular on periodic KYC updation. RegIntel parses the PDF, extracts each binding obligation (e.g. "REs shall ensure periodic updation of KYC records within the prescribed timelines"), embeds it, and searches the bank's indexed internal policy library. The obligation has no closely matching policy, so it is flagged as a gap, scored against historical KYC/AML enforcement severity and the stated deadline, and ranked Critical in the generated PDF report.

Scenario 2: A risk team runs a batch audit across several SEBI and FEMA circulars at once. The orchestration agent processes each PDF in sequence — parsing, extracting obligations, checking policy coverage, and scoring gaps — and every run is written to the SQLite audit log, so the team can later compare gap counts and average penalty exposure across circulars in the Audit History view.

Scenario 3: During extraction, the Groq model returns zero obligations on the first pass for a long, densely worded circular. Rather than accepting this silently, the orchestration agent recognizes the circular text was substantial and re-runs extraction once before finalizing the result — catching obligations that a single deterministic pass would have missed.

Scenario 4: A bank has not yet uploaded any internal policy documents for a particular business line. When the gap-check stage runs, RegIntel explicitly recognizes that no policies are indexed and notes this as a caveat in the audit log and agent summary, rather than letting every obligation default to a gap without explanation — giving the compliance team an honest signal to index their policies first.

Technical Architecture:



Pre-requisites:

- **Python Programming Proficiency: Python Documentation**
  https://docs.python.org/3/
- **Flask Framework Knowledge: Flask Documentation**
  https://flask.palletsprojects.com/
- **LangChain & Agentic Orchestration Familiarity: LangChain Documentation**
  https://python.langchain.com/docs/
- **Groq API Familiarity: Groq API Documentation**
  https://console.groq.com/docs
- **ChromaDB & Vector Database Basics: ChromaDB Documentation**
  https://docs.trychroma.com/

- **Sentence-Transformers for Semantic Embeddings: Sentence-Transformers Documentation**
  https://www.sbert.net/

- **PyMuPDF PDF Processing Library: PyMuPDF Documentation**
  https://pymupdf.readthedocs.io/

- **ReportLab PDF Generation Library: ReportLab Documentation**
  https://docs.reportlab.com/

- **SQLite Database Basics: SQLite Documentation**
  https://www.sqlite.org/docs.html

- **HTML, CSS, and JavaScript Skills: W3Schools Tutorials**
  https://www.w3schools.com/

- **Version Control with Git: Git Documentation**
  https://git-scm.com/doc

- **Development Environment Setup: Conda / Miniconda Documentation**
  https://docs.conda.io/

Project Workflow:

Activity 1: Model Selection and Architecture

- Activity 1.1: Generate a Groq API key and configure it securely in the project environment.
- Activity 1.2: Research and select the appropriate generative AI model from Groq for obligation extraction and agentic tool-calling.
- Activity 1.3: Define the architecture of the application, detailing how the LangChain AgentExecutor orchestrates the parse, extract, gap-check, and score stages.
- Activity 1.4: Set up the development environment, installing the LangChain, ChromaDB, Sentence-Transformers, PyMuPDF, ReportLab, and Flask dependencies.

Activity 2: Core Pipeline Development

- Activity 2.1: Develop the core pipeline stages: PDF ingestion, obligation extraction, policy gap detection, and penalty scoring.
- Activity 2.2: Implement the LangChain AgentExecutor with four tool-calling functions, ensuring the agent can retry or escalate at each decision point.

Activity 3: Orchestrator & Flask Backend Development

- Activity 3.1: Write the agent orchestration logic in orchestrator.py, defining tools for each pipeline stage and the prompt that governs their execution order.

Activity 4: Frontend Development

- Activity 4.1: Design and develop the Audit Console user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- Activity 4.2: Build dynamic rendering of the agent trace, gap analysis table, and audit history charts based on live API responses.

Activity 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the conda environment and ensuring all dependencies are correctly installed.
- Activity 5.2: Run a Groq model health check at startup and verify the full audit pipeline against a sample circular before serving real requests.

Activity 6: Conclusion

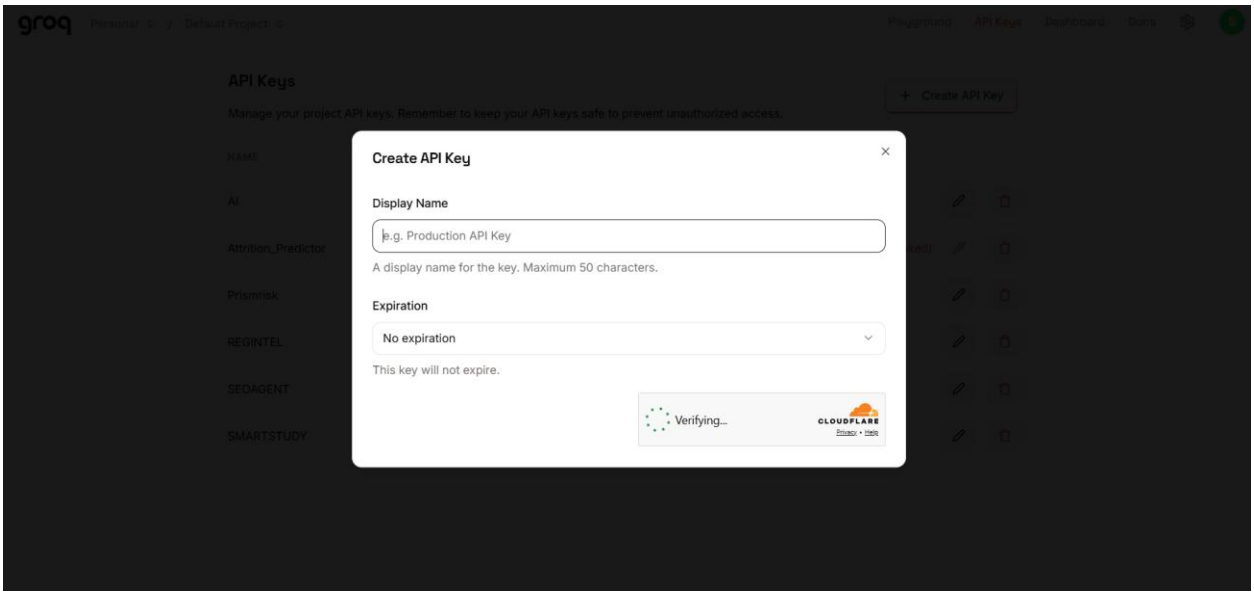
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for obligation extraction and agentic orchestration. This involves researching the tool-calling compatibility and reasoning behaviour of candidate models, since the AgentExecutor requires clean, parseable tool-call output at every stage of the pipeline.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq-hosted LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- Step 1 — Create a Groq Account: Visit https://console.groq.com and sign up for a free account using your Google account or email address.
- Step 2 — Navigate to API Keys: After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.
- Step 3 — Create a New API Key: Click the "Create API Key" button. Give your key a descriptive name (e.g., regintel-dev) so it is easy to identify later.



- Step 4 — API Key: Give a "Display Name" and leave "Expiration" as optional, then click "Submit". Copy the key immediately and store it in a safe place — you will not be able to view it again after closing the dialog.

Step 5 — Add the Key to Your Project: Copy .env.example to .env in the project root and paste the key as shown below:

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
```

```
CHROMA_PERSIST_DIR=data/chroma_db
EMBEDDING_MODEL=all-MiniLM-L6-v2
```

Step 6 — Verify the Key is Loaded: The config.py module loads this key automatically at startup using python-dotenv, and raises a clear error if it is missing or still set to the placeholder value:

```
# Load variables from a .env file in the project root, if present.
load_dotenv(override=True)

PROJECT_ROOT = Path(__file__).resolve().parent
DATA_DIR = PROJECT_ROOT / "data"
CIRCULARS_DIR = DATA_DIR / "circulars"
POLICIES_DIR = DATA_DIR / "policies"
REPORTS_DIR = PROJECT_ROOT / "outputs" / "reports"

for _dir in (CIRCULARS_DIR, POLICIES_DIR, REPORTS_DIR):
    _dir.mkdir(parents=True, exist_ok=True)
```

Important — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

- Understand the Project Requirements: Review the specific needs of the RegIntel pipeline — reliable tool-calling for the AgentExecutor, deterministic JSON-style extraction of obligation clauses, and stable behaviour across RBI, SEBI, and FEMA circular text.
- Explore Groq's Model Documentation: Visit the Groq documentation to examine available LLM models, including tool-calling compatibility, context windows, and inference speed.
- Evaluate Model Performance: Compare candidate models based on tool-call format compatibility with LangChain's create_tool_calling_agent, extraction accuracy on regulatory text, and generation speed.
- Select the Optimal Model: Choose llama-3.3-70b-versatile, set with a temperature of 0.0 for deterministic extraction and reasoning_effort set to "none" to suppress reasoning-mode preambles that would otherwise break strict-JSON and tool-call parsing.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual representation of the application architecture, including the LangChain AgentExecutor, its four tools, the Flask backend, and the dashboard frontend.
- Detail Frontend Functionality: Outline how users interact with the application — uploading a circular PDF (or batch of PDFs), watching the live agent trace step through parsing, extraction, gap-checking, and scoring, and reviewing the ranked gap table and downloadable PDF report.
- Outline Backend Responsibilities: Specify how the Flask backend handles incoming POST requests to /api/audit, saves uploaded PDFs to a temporary directory, and invokes RegIntelPipeline.run_batch() to process them.
- Describe Agentic Integration Points: Define how the agent decides, at each of the four pipeline stages, whether to proceed, retry, or flag a caveat — rather than following one fixed deterministic path regardless of intermediate results.

Activity 1.4: Set Up the Development Environment

- Install Anaconda or Miniconda: Ensure conda is installed on your machine for environment management.
- Create a Conda Environment: Set up an isolated environment using conda:

```
conda create -n regintel python=3.10 -y
conda activate regintel
```

- Install All Dependencies: Everything needed is pinned in requirements.txt and installed in one shot:

```
pip install -r requirements.txt
```

- Configure the API Key: Copy .env.example to .env in the project root and add your Groq key as shown in Activity 1.1.
- Set Up Application Structure: Create the project directory structure including folders for src (ingestion, extraction, vectorstore, scoring, report, agent, storage, utils), data (circulars, policies, chroma_db), outputs/reports, frontend (HTML, CSS, JS), and the main application files (app.py, flask_app.py, run_audit.py, config.py, requirements.txt).

Milestone 2: Core Pipeline Development

Activity 2.1: Develop Core Pipeline Stages

Implement the four core pipeline stages — PDF Ingestion, Obligation Extraction, Gap Detection, and Penalty Scoring — as independent, testable modules that the agent later orchestrates as tools.

- PDF Ingestion (src/ingestion/pdf_parser.py): Uses PyMuPDF to extract full text page-by-page from a circular PDF, then applies regex patterns to detect the regulator (RBI, SEBI, or FEMA), the circular number, and the circular date from the first two pages. Raises a clear ValueError if the PDF has no extractable text, since this pipeline does not include OCR.

```
def parse_pdf(pdf_path: str | Path) -> ParsedCircular:
    """Parse a single regulatory circular PDF into structured text + metadata.

    Raises:
        FileNotFoundError: if pdf_path does not exist.
        ValueError: if the PDF has no extractable text (e.g. scanned image
            without OCR), since obligation extraction requires real text.
    """
    pdf_path = Path(pdf_path)
    if not pdf_path.exists():
        raise FileNotFoundError(f"PDF not found: {pdf_path}")

    pages: list[str] = []
    with fitz.open(pdf_path) as doc:
        for page in doc:
            pages.append(page.get_text("text"))
        page_count = doc.page_count

    full_text = "\n".join(pages).strip()
    if not full_text:
        raise ValueError(
            f"No extractable text found in {pdf_path.name}. "
            f"This file may be a scanned image requiring OCR, which is "
            f"outside RegIntel's current free-tier pipeline."
        )
```

- Obligation Extraction (src/extraction/obligation_extractor.py): Chunks long circular text into ~6000-character LLM-context-friendly segments, sends each chunk to Groq's Llama 3.3-70B model with a system prompt instructing it to return a strict JSON array of binding obligation clauses, and parses the result into typed Obligation records (clause_text, obligation_type, deadline_text, applicable_entity).

```
_SYSTEM_PROMPT = dedent(
    """\
    You are a regulatory compliance analyst for an Indian bank. Your job is
    to read excerpts of RBI/SEBI/FEMA circulars and extract every BINDING
    OBLIGATION clause — language such as "shall", "must", "is required to",
    or "within N days" that creates a concrete duty for a regulated entity.

    For each obligation found, return a JSON object with these exact keys:
      - "clause_text": the obligation sentence, verbatim, max 60 words
      - "obligation_type": one of
          ["reporting", "disclosure", "capital_liquidity", "kyc_aml",
           "governance", "deadline", "other"]
      - "deadline_text": the deadline phrase if stated, else null
      - "applicable_entity": who must comply, e.g. "Scheduled Commercial
          Banks", "NBFCs", "AIFs" — null if not specified

    Respond with ONLY a JSON array of such objects, no commentary, no
    markdown code fences. If no obligations are present, return [].
    """
)
```

- Gap Detection (src/vectorstore/chroma_manager.py): Embeds extracted obligations and internal policy chunks into a local, persistent ChromaDB instance using Sentence-Transformers (CPU-only, free), then queries the policy collection for the closest semantic match to each obligation — flagging any obligation whose best match falls below a similarity threshold of 0.55 as a compliance gap.

```
def find_gaps(self, obligations: list[Obligation], *, similarity_threshold: float = 0.55) -> list[GapMatch]:
    """For every provided obligation, search the policy collection for the
    closest semantic match. If the best match falls below the similarity
    threshold (or no policies exist at all), the obligation is flagged
    as an uncovered compliance gap.
    """
    results: list[GapMatch] = []
    has_policies = self.policies.count() > 0

    for ob in obligations:
        doc_id = _stable_id(ob.clause_text, ob.circular_number or "")
        meta = {
            "obligation_type": ob.obligation_type,
            "deadline_text": ob.deadline_text or "",
            "applicable_entity": ob.applicable_entity or "",
            "circular_number": ob.circular_number or "",
            "regulator": ob.regulator or "",
            "source_path": ob.source_path or "",
        }

        if not has_policies:
            results.append(
                GapMatch(
                    obligation={"id": doc_id, "clause_text": ob.clause_text, **meta},
                    best_policy_match=None,
                    similarity_score=0.0,
                    is_gap=True,
                )
            )
            continue
```

- Penalty Scoring (src/scoring/penalty_scorer.py): Combines three weighted signals into a single 0–100 penalty exposure score for every detected gap — historical RBI/SEBI enforcement severity by obligation type, deadline proximity parsed from the obligation's stated deadline text, and policy staleness derived from how semantically distant the closest internal policy is — then ranks gaps into Critical, High, Medium, and Low risk bands.

```
_SEVERITY_WEIGHTS: dict[str, int] = {
    "capital_liquidity": 60,
    "kyc_aml": 55,
    "reporting": 45,
    "disclosure": 40,
    "governance": 35,
    "deadline": 50,
    "other": 25,
}

_DAYS_PATTERN = re.compile(r"(\d+)\s*(?:calendar\s*)?days?", re.IGNORECASE)
_WEEKS_PATTERN = re.compile(r"(\d+)\s*weeks?", re.IGNORECASE)
_MONTHS_PATTERN = re.compile(r"(\d+)\s*months?", re.IGNORECASE)
_IMMEDIATE_PATTERN = re.compile(r"\b(?:immediate(?:ly)?|forthwith|with immediate effect|come into effect immediately)\b", re.IGNORECASE)

_WORD_NUMBERS = {
    "one": 1, "two": 2, "three": 3, "four": 4, "five": 5,
    "six": 6, "seven": 7, "eight": 8, "nine": 9, "ten": 10,
    "fifteen": 15, "thirty": 30, "sixty": 60, "ninety": 90
}
```

Activity 2.2: Implement Deadline Parsing and Risk Banding

parse_deadline_days(): a defensive parser that converts a circular's free-text deadline ("within 30 days", "immediately", "by 31 March 2026") into a concrete day count, falling back through regex patterns for days/weeks/months, a word-number translation table, and finally dateutil's fuzzy date parser before giving up and returning None.

```
def parse_deadline_days(deadline_text: str | None) -> int | None:
    if not deadline_text:
        return None
    text = deadline_text.strip().lower()
    if _IMMEDIATE_PATTERN.search(text):
        return 0
    for word, num in _WORD_NUMBERS.items():
        text = re.sub(rf"\b{word}\b", str(num), text)
    match = _DAYS_PATTERN.search(text)
    if match:
        return int(match.group(1))
    match = _WEEKS_PATTERN.search(text)
    if match:
        return int(match.group(1)) * 7
    match = _MONTHS_PATTERN.search(text)
    if match:
        return int(match.group(1)) * 30
    try:
        clean_text = re.sub(r'^(?:by|on or before|until|from)\s+', '',
deadline_text.strip(), flags=re.IGNORECASE)
        dt = parse_date(clean_text, fuzzy=True)
        if dt.tzinfo is not None:
```

```
            dt = dt.replace(tzinfo=None)
        return (dt.date() - datetime.now().date()).days
    except (ValueError, TypeError, OverflowError):
        pass
    return None
```

_risk_band(): maps the combined 0–100 penalty score into one of four risk bands, used throughout the dashboard and PDF report to colour-code remediation priority.

```
def _risk_band(score: float) -> str:
    if score >= 75:
        return "Critical"
    if score >= 55:
        return "High"
    if score >= 35:
        return "Medium"
    return "Low"
```

_SEVERITY_WEIGHTS: an indicative historical enforcement severity table, derived from publicly disclosed RBI/SEBI penalty orders, expressed as a base score out of 60 (the remaining 40 points come from deadline urgency and policy staleness).

```
_SEVERITY_WEIGHTS: dict[str, int] = {
    "capital_liquidity": 60,
    "kyc_aml": 55,
    "reporting": 45,
    "disclosure": 40,
    "governance": 35,
    "deadline": 50,
    "other": 25,
}
```

Milestone 3: Orchestrator and Flask Backend Development

Milestone 3 focuses on creating and refining the agent orchestration logic in src/agent/orchestrator.py, which serves as the backbone of the RegIntel pipeline, and the Flask backend in flask_app.py that exposes it over a REST API. In this milestone, you'll wire up the four pipeline stages as LangChain tools, define the prompt that governs their execution order, and expose audit, history, and report-download endpoints.

Activity 3.1: Building the LangChain AgentExecutor

Rather than a fixed deterministic pipeline, RegIntel uses a LangChain tool-calling agent that decides at runtime how to handle four points where a hardcoded script would otherwise have to guess: a parsing failure or near-empty extracted text, zero obligations extracted on the first pass, no internal policies indexed yet, and borderline similarity scores between 0.50 and 0.60.

```
def _build_agent(self) -> AgentExecutor:
    llm = ChatGroq(
        api_key=self.settings.groq_api_key,
        model=self.settings.groq_model,
        temperature=0.0,
        # See obligation_extractor._build_llm for why this is set: models
        # like qwen/qwen3.6-27b default to "thinking mode" and prepend a
        # <think>...</think> block, which breaks LangChain's tool-call
        # parsing. Harmless to send to models that don't support it.
        model_kwargs={"reasoning_effort": "none"},
    )

    tools = [
        self._make_parse_tool(),
        self._make_extract_tool(),
        self._make_gap_check_tool(),
        self._make_score_tool(),
    ]
```

```
prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are the orchestration agent for RegIntel, a regulatory "
            "compliance gap-detection pipeline for an Indian bank. You "
            "are given the path to one circular PDF and must run it "
            "through all four stages, in order: parse_circular, "
            "extract_obligations_tool, check_policy_coverage, "
            "score_gaps_tool. Always call them in that order, exactly "
            "once each, UNLESS a tool's result tells you to retry or "
            "stop early (the tool descriptions explain when). After "
            "parse_circular, if it reports near-empty or failed "
            "extraction, retry it at most once before giving up. "
            "After extract_obligations_tool, if zero obligations were "
            "found, call it a second time only if the circular text "
            "looked substantial (run it again with the same input — "
            "do not skip it just because the count was zero unless "
            "you've already retried once). When you finish all "
            "stages, respond with a short plain-text summary of what "
            "you did and any concerns, then stop. Be decisive and "
            "concise; this is a compliance tool, not a chat assistant.",
        ),
        ("human", "{input}"),
        MessagesPlaceholder(variable_name="agent_scratchpad"),
    ]
)
```

Activity 3.2: Defining the Four Orchestration Tools

- parse_circular: parses the circular PDF and reports back page count, character count, and a thin_extraction flag. If the agent sees thin_extraction is true, it may retry this tool once before giving up.
- extract_obligations_tool: extracts obligations from the most recently parsed circular and reports the count found. If zero obligations come back on a substantial circular, the agent may call it a second time.
- check_policy_coverage: embeds the obligations, searches the policy collection for semantic matches, and explicitly reports whether no policies are indexed at all — so the agent can mention this caveat rather than silently flag everything as a gap.
- score_gaps_tool: scores and ranks all detected gaps, then generates the PDF report. This is always called last and finalises the audit run.

check_policy_coverage tool implementation:

```
@tool(args_schema=_CheckPolicyCoverageInput)
def check_policy_coverage() -> str:
    state = self._state
    obligations = state.get("obligations", [])
    self.vector_store.add_obligations(obligations)
    gaps = self.vector_store.find_gaps(obligations)
    state["gaps"] = gaps

    no_policies = self.vector_store.policies.count() == 0
    borderline = [
        g for g in gaps
```

```
        if self._BORDERLINE_LOW <= g.similarity_score < self._BORDERLINE_HIGH
    ]
    return json.dumps({
        "ok": True,
        "obligations_checked": len(gaps),
        "gaps_flagged": sum(1 for g in gaps if g.is_gap),
        "borderline_count": len(borderline),
        "no_policies_indexed": no_policies,
    })
```

Activity 3.3: Implementing the Flask Backend

Define Routes in flask_app.py: set up routes for serving the dashboard, running an audit, retrieving audit history, and downloading a generated PDF report.

```
@app.route("/")
def index():
    return send_from_directory("frontend", "index.html")

@app.route("/api/audit", methods=["POST"])
def api_audit():
    if not pipeline:
        return jsonify({"error": "Pipeline not initialized..."}), 500

    uploaded_files = request.files.getlist("file") or request.files.getlist("files")
    if not uploaded_files or all(f.filename == "" for f in uploaded_files):
        return jsonify({"error": "No files uploaded."}), 400

    with tempfile.TemporaryDirectory() as tmp_dir:
        tmp_paths = [Path(tmp_dir) / f.filename for f in uploaded_files]
        for f, p in zip(uploaded_files, tmp_paths):
            f.save(str(p))
        results = pipeline.run_batch(tmp_paths)
    return jsonify({"results": [...]})
```

Startup Health Check: before the real pipeline is built, flask_app.py runs a lightweight tool-calling round trip against the configured Groq model. This catches a deprecated model or an incompatible tool-call format at startup with a clear, specific message, instead of as an opaque 500 error the first time someone submits a real audit.

```
health = check_groq_model(settings)
if not health.ok:
    print("GROQ HEALTH CHECK FAILED — pipeline will NOT be initialized.")
    print(health.message)
else:
    pipeline = RegIntelPipeline(settings)
```

- Process Uploaded Files: accept one or more multipart PDF uploads, validate that each filename ends in .pdf, save them to a temporary directory, and pass the batch to RegIntelPipeline.run_batch().
- Expose Audit History: the /api/history endpoint returns real past audit run data from the SQLite audit log via get_audit_history(), including obligation counts, gap counts, average penalty score, and per-risk-band counts for each run.
- Expose Report Downloads: the /api/report/<circular_number> endpoint resolves a circular number or filename to the corresponding generated PDF in outputs/reports/, trying direct filename matching and circular-number-to-filename mapping as fallback strategies.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a professional, audit-console-style interface for RegIntel. This involves building a marketing landing page that explains the regulatory scope and technology stack, a live audit console with drag-and-drop PDF upload, an animated agent trace that mirrors the four pipeline stages as they execute, and a sortable audit history view backed by real Chart.js visualisations.

Activity 4.1: Designing and Developing the User Interface

- Set Up the Base HTML Structure: develop the main interface (frontend/index.html) with a navigation bar, hero section describing the regulatory scope (RBI, SEBI, FEMA) and technology stack (Llama 3 via Groq, LangChain, ChromaDB), and a Dashboard section containing the audit console.
- Use Semantic HTML Elements: keep the structure organised and ensure the interface is accessible, with Lucide icons for navigation, status indicators, and step markers, and Google Fonts (Inter, Playfair Display, JetBrains Mono) for typography.
- Design a Responsive Layout Using CSS: implement the dark-theme design system in frontend/styles.css with light/dark theme toggling, using flexbox and grid layouts to organise the agent trace, charts, and results table.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>RegIntel | Automated Compliance Intelligence for Indian Banks</title>
    <meta name="description" content="RegIntel is an agentic AI platform that automatically extracts binding obligations from RBI/SEBI regulatory ci

    <!-- Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&family=Playfair+Display:wght@400;500;600;700&family=JetBrains+Mono:w

    <!-- Icons & Charts -->
    <script src="https://unpkg.com/lucide@latest"></script>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

    <!-- Styles -->
    <link rel="stylesheet" href="styles.css">
</head>
<body class="dark-theme">
    <!-- Navigation -->
    <nav class="navbar" id="navbar">
        <div class="nav-container">
            <a href="#dashboard" class="logo">
                <i data-lucide="shield-check" class="logo-icon"></i>
                <span>RegIntel</span>
            </a>
            <div class="nav-links">
                <a href="#platform">Platform</a>
                <a href="#dashboard">Audit Console</a>
                <a href="#cases">Use Cases</a>
```

Activity 4.2: Building the Audit Console

- Upload Zone: a drag-and-drop area accepting PDF circulars, with a hidden file input and a “Browse files” button as a fallback interaction.

- Processing Zone (Agent Trace): four sequential trace steps — Parsing document, Extracting obligation clauses, Searching policy vector store, and Scoring compliance gaps — each with a spinner that resolves to a checkmark as that stage of the real pipeline completes, mirroring the four LangChain tools defined in the orchestrator.
- Results Zone: a Gap Analysis Report header with a status badge and a Download PDF Report button, two real Chart.js visualisations (Gaps by Severity and Top Priority Gaps), and a results table listing each obligation clause, its risk band, penalty score, and coverage status.
- Error Zone: a clear error state with the specific failure message returned by the API (e.g. a Groq health-check failure or an unparseable PDF), with a "Try another audit" action to reset the console.

Activity 4.3: Building the Audit History View

- Tabbed Navigation: a "Run Audit" / "Audit History" tab switcher above the console, so compliance officers can move between submitting a new circular and reviewing past runs without leaving the page.
- History Timeline Chart: a Chart.js line chart plotting gaps detected over time across all logged audit runs, fetched from /api/history.
- Sortable History Table: columns for timestamp, circular number, regulator, obligations found, gaps detected, and average penalty score, each independently sortable, plus a per-row link to download that run's generated PDF report.

Activity 4.4: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously: attach a submit handler to the upload zone that sends the selected PDF(s) as multipart/form-data to /api/audit via the Fetch API, transitioning the UI from the Upload Zone to the Processing Zone.

- Animate the Agent Trace: step through each of the four trace markers in sequence as the request is in flight, giving the compliance officer visibility into which pipeline stage is currently running.
- Render Results Dynamically: on a successful response, populate the Chart.js severity and priority charts and inject table rows for every scored gap returned by the API, then transition to the Results Zone.
- Render Errors Gracefully: on a failed response, surface the specific error message from the API in the Error Zone rather than a generic failure notice.
- Restore UI State: provide a reset action on both the Results Zone and Error Zone that returns the console to the Upload Zone, ready for the next circular.

Milestone 5: Local Testing and Deployment

In Milestone 5, the focus is on preparing RegIntel for local deployment, verifying correct operation against a sample circular, and running the offline unit test suite. Because the pipeline depends on the Groq API and a persistent local vector store rather than a hosted server, deployment here means a fully reproducible local environment rather than a public tunnel.

Activity 5.1: Preparing the Application for Local Deployment

Create and Activate the Conda Environment:

```
conda create -n regintel python=3.10 -y
conda activate regintel
pip install -r requirements.txt
```

Configure Environment Variables: copy .env.example to .env in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
```

- Add Internal Policy Documents (optional but recommended): drop the bank/NBFC's internal SOP or policy documents as plain .txt files into data/policies/ so the gap-check stage has something real to compare obligations against.
- Add a Sample Circular: a bundled regulatory circular PDF can be placed in data/circulars/ as a quick smoke-test asset before processing real, unreleased circulars.

Activity 5.2: Running the Flask Backend Locally

Start the Flask Backend:

```
python flask_app.py
```

```
PS C:\Users\baiya\OneDrive\Documents\Projects\regintel> python flask_app.py
Loading Settings...
Running Groq health check against model 'qwen/qwen3.6-27b'...
Groq model 'qwen/qwen3.6-27b' passed the tool-calling health check.
Initializing RegIntelPipeline...
Failed to send telemetry event ClientStartEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argument but 3 were given
RegIntelPipeline successfully initialized.
 * Serving Flask app 'flask_app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [26/Jun/2026 07:21:54] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [26/Jun/2026 07:21:54] "GET /styles.css HTTP/1.1" 200 -
127.0.0.1 - - [26/Jun/2026 07:21:54] "GET /app.js HTTP/1.1" 200 -
```

- Once running, the startup health check validates the configured Groq model before the pipeline is built — watch the terminal for either "RegIntelPipeline successfully initialized" or a specific health-check failure message.
- Open a browser and navigate to http://127.0.0.1:5000 to access the Audit Console.
- Upload a circular PDF, watch the agent trace step through parsing, extraction, gap-checking, and scoring, and confirm the resulting gap table, charts, and downloadable PDF report match what the pipeline logged to the terminal.
- Switch to the Audit History tab and confirm the run just completed appears with the correct obligation count, gap count, and average penalty score.

Activity 5.3: Running the CLI Entry Point

As an alternative to the dashboard, the pipeline can be run directly from the command line:

```
python run_audit.py data/circulars/sample_circular.pdf
```

This prints a ranked gap summary to the terminal and saves a PDF report to outputs/reports/, using the exact same RegIntelPipeline used by the Flask backend — so behaviour is identical whether the audit is triggered from the API or the CLI.

Activity 5.4: Running the Test Suite

Pure-logic unit tests for the penalty scoring module run instantly with no API calls, since they exercise score_and_rank_gaps() and parse_deadline_days() directly against fixed inputs:

```
pip install pytest
pytest tests/ -v
```

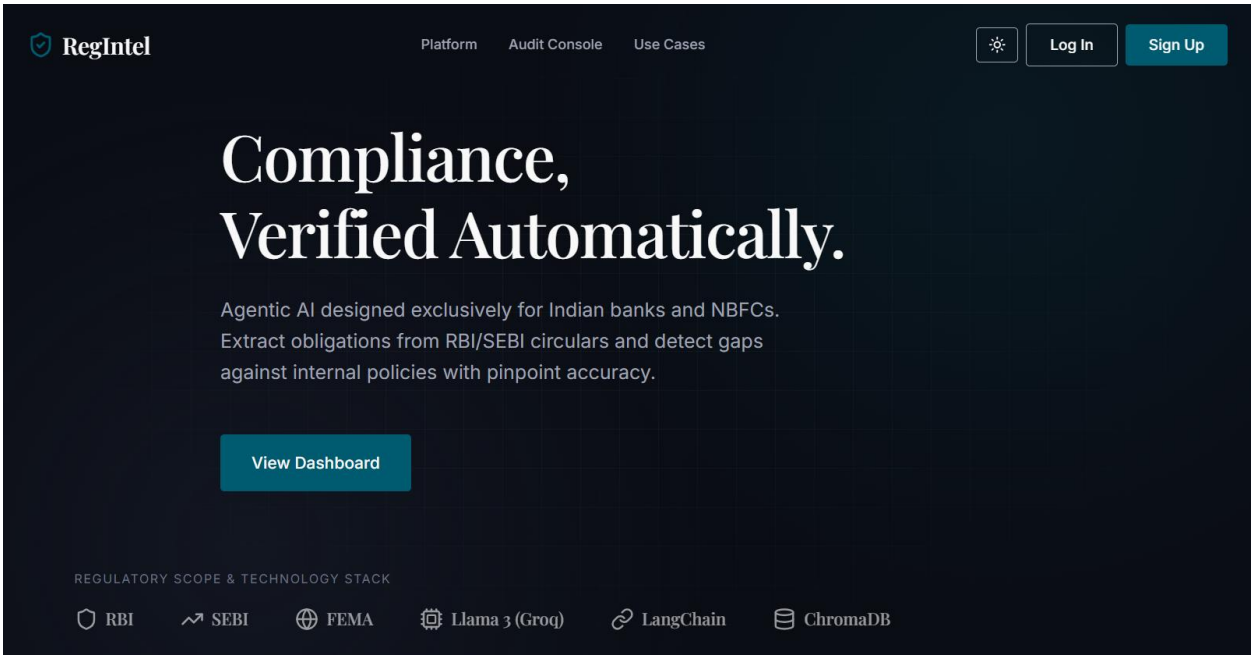
Important Notes:

- Free-tier Groq rate limits: the obligation extractor already retries with exponential backoff (tenacity); for very large circulars, consider splitting into smaller PDFs if rate limit errors persist.
- Resetting the vector database: delete the data/chroma_db/ folder and re-index policies / re-run an audit — it will be recreated automatically on next use.
- OCR is intentionally out of scope: a scanned-image PDF with no extractable text will raise a clear ValueError at parse time rather than silently producing zero obligations.
- Deterministic-but-agentic design: every run executes the same four stages in the same order, but the agent — not a hardcoded script — decides whether to retry, escalate, or proceed at each of the four decision points, while every tool call is still logged for auditability.

Exploring the Website's Web Pages:

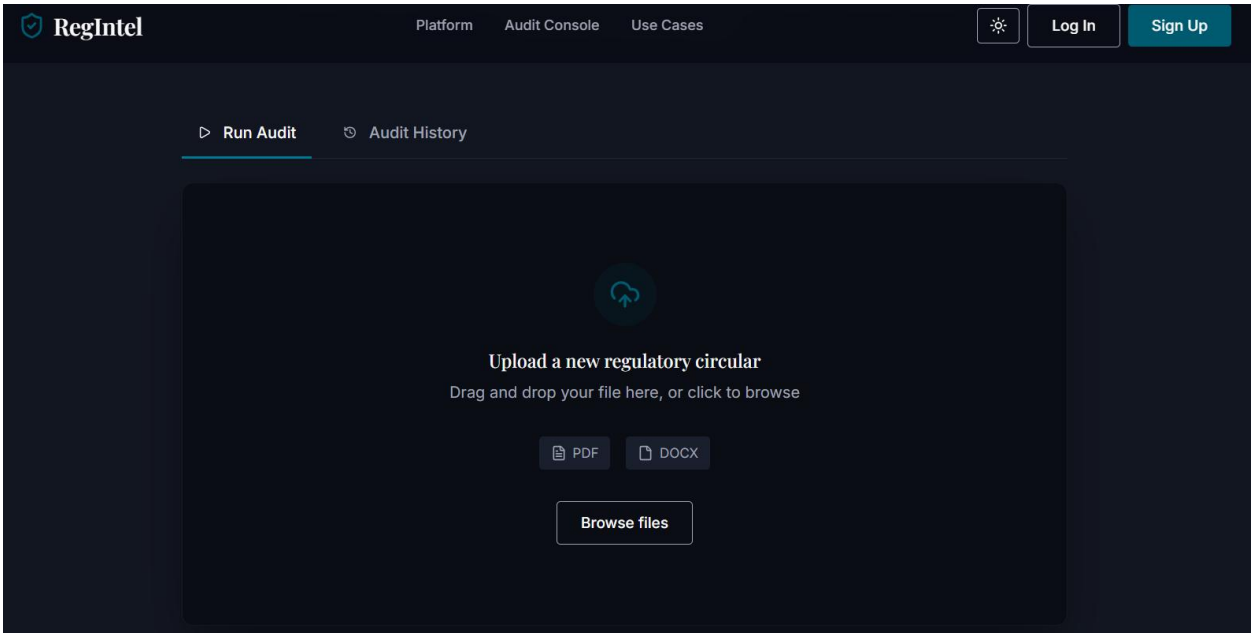
Landing / Dashboard Page:

The single-page interface of RegIntel serves as both an explanatory landing page and the live audit console. It features a dark theme with a light-mode toggle, a navigation bar displaying the RegIntel brand (shield-check icon + wordmark), and a hero section stating the platform's scope and tagline. Below the hero, a logo strip lists the regulatory scope (RBI, SEBI, FEMA) and the technology stack (Llama 3 via Groq, LangChain, ChromaDB) the platform runs on.



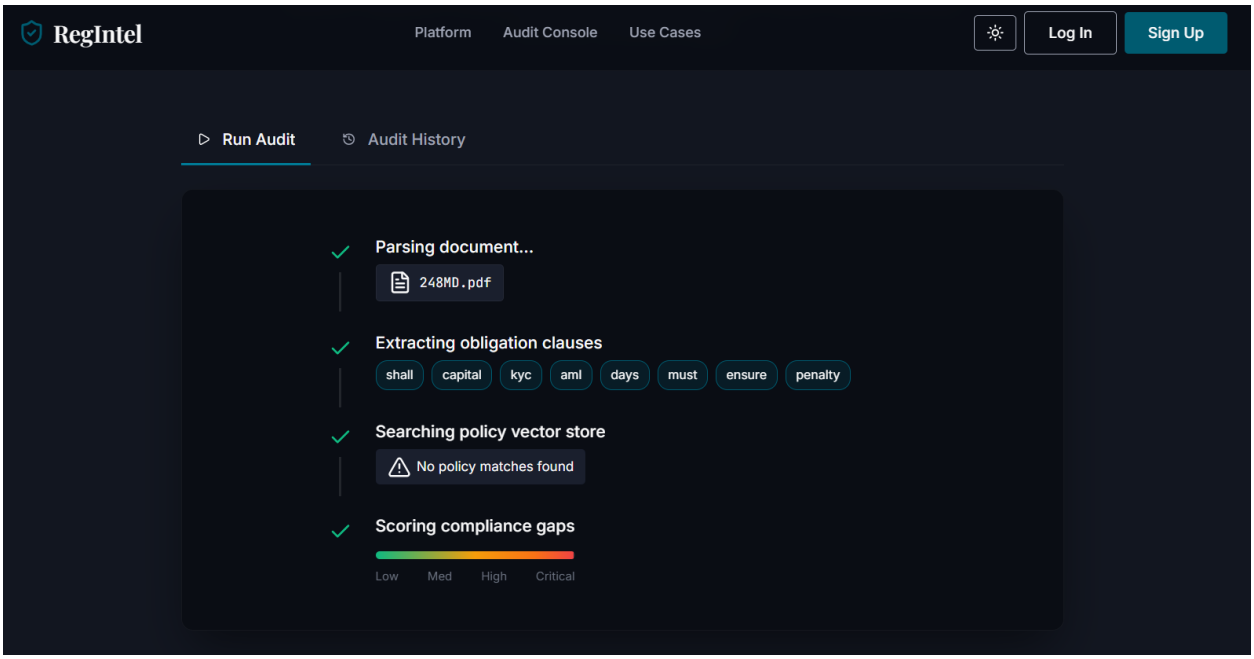
Audit Console — Upload State:

A bordered drop zone inviting the compliance officer to drag and drop a circular PDF, or browse for one via a hidden file input. Accepted-format badges indicate PDF and DOCX support, and a tab switcher above the console lets the user move between "Run Audit" and "Audit History" without leaving the page.



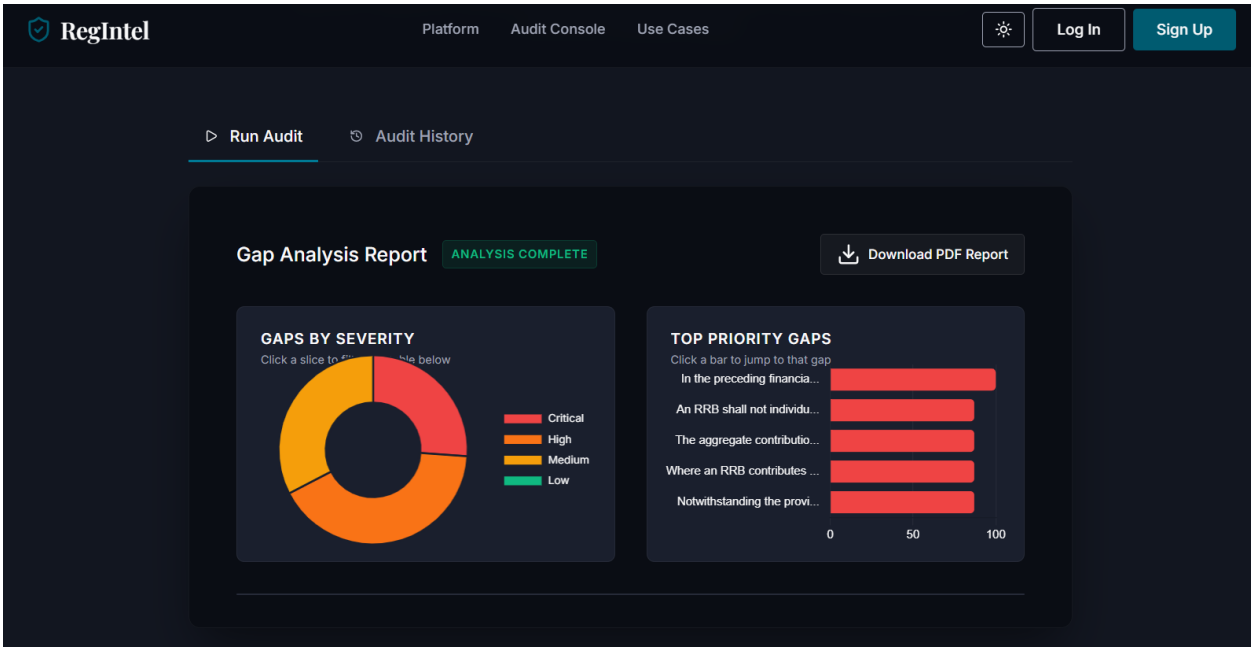
Audit Console — Processing State:

Once a file is submitted, the console reveals an agent trace with four sequential steps — parsing the document, extracting obligation clauses, searching the policy vector store, and scoring compliance gaps — each marked with a spinner that resolves to a checkmark as that real pipeline stage completes. A risk meter bar with Low / Medium / High / Critical labels animates alongside the scoring step.



Audit Console — Results State:

Revealed after a successful audit, this section displays a Gap Analysis Report header with a success status badge and a Download PDF Report button. Two Chart.js visualisations — Gaps by Severity and Top Priority Gaps — sit above a results table listing each obligation clause, its risk band, penalty score, and coverage status (Policy Gap Detected, Aligned, or Requires Review). An “Analyze another document” action resets the console to the upload state.



Downloadable in PDF Format:

RegIntel — Compliance Gap Report

Audit of 248MD.pdf (RBI)
Generated: 26 June 2026, 08:02

Total gaps detected: **46**

#	Risk	Score	Regulator	Circular	Obligation Clause	Deadline	Recommended Remediation Action
1	Critical	100.0	RBI	—	In the preceding financial year, an RRB shall have: A net worth of ■100 crore or more as on March 31	as on March 31	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures. Target compliance window: as on March 31.
2	Critical	87.0	RBI	—	An RRB shall not individually contribute more than 10 percent of the corpus of an AIF Scheme.	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
3	Critical	87.0	RBI	—	The aggregate contribution by all Regulated Entities (REs) in any AIF Scheme shall not be more than 20 percent of the corpus of that scheme.	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
4	Critical	87.0	RBI	—	Where an RRB contributes more than five percent of the corpus of an AIF Scheme that has downstream investment... the RRB shall be required to make 100 percent provision...	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
5	Critical	87.0	RBI	—	Notwithstanding the provisions of paragraph 13, where a RE's contribution is in the form of subordinated units, it shall deduct the entire investment from its capital funds...	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
6	Critical	87.0	RBI	—	In the preceding financial year, an RRB shall have: A Capital to Risk-Weighted Assets Ratio (CRAR) of not less than nine percent	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
7	Critical	87.0	RBI	—	In the preceding financial year, an RRB shall have: Net NPAs below five percent.	—	Update capital adequacy, liquidity buffers, and assets ratio calculations/procedures.
8	Critical	85.0	RBI	—	Statement for the half year ended March …. / September …. (to be submitted by 10th of following month)	by 10th of following month	Establish automated or manual data queries, review cycles, and submission templates to send reports to the regulator. Target compliance window: by 10th of following month.
9	Critical	82.0	RBI	—	An RRB shall comply with the extant KYC / AML guidelines in respect of the applicants.	—	Update KYC/AML policy, verification checklists, and client onboarding systems to align with this requirement.

Audit Console — Error State:

If the pipeline fails — for example, a Groq health-check failure at startup or a scanned PDF with no extractable text — this section surfaces the specific error message returned by /api/audit, with a "Try another audit" action rather than a generic failure notice.

```
PS C:\Users\baiya\OneDrive\Documents\Projects\regintel> python flask_app.py
Loading Settings...
Running Groq health check against model 'qwen/qwen3.6-27b'...
Groq model 'qwen/qwen3.6-27b' passed the tool-calling health check.
Initializing RegIntelPipeline...
```

Audit History Page:

Switching to the Audit History tab reveals a timeline chart plotting gaps detected over time across all logged runs, and a sortable table of every past audit — timestamp, circular number, regulator, obligations found, gaps detected, average penalty score, and a per-row link to re-download that run's generated PDF report — all backed by the real SQLite audit log rather than placeholder data.

Conclusion:

RegIntel is an agentic regulatory compliance platform built with Flask, LangChain, and a local ChromaDB vector store that automates one of the most labour-intensive tasks in Indian banking compliance: reading dense RBI, SEBI, and FEMA circulars and checking every binding obligation against existing internal policy coverage. It uses Groq's high-speed inference API with the Llama 3.3-70B Versatile model, orchestrated through a LangChain tool-calling AgentExecutor, to extract obligations, detect coverage gaps, and score penalty exposure — with the agent itself deciding when to retry a thin extraction, re-run a suspiciously empty result, or flag a missing policy library, rather than following one brittle deterministic path. The project, which included model selection, agentic pipeline development, a Flask REST backend, a live audit-console frontend, and a persistent SQLite audit log, highlights how a structured, free-tier AI stack can give compliance teams an auditable, board-ready gap report in minutes, with clear room for future scalability toward OCR support and multi-institution deployments.